

Task 2 — DTO, Validation & Global Exception Handling

Step 1 — Understand DTOs

Entity

An **Entity** represents your database table.

Example:

@Entity

@Table(name = "users")

```
public class User {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

```
    private String email;
```

```
    private String password;
```

```
}
```

This class is connected to the database.

For example, the database might contain:

id	name	email	password
1	Ravi	ravi@gmail.com	abc123

DTO (Data Transfer Object)

A DTO is a separate Java class used to transfer data between the client and your application.

Instead of exposing the Entity:

User Entity

↓

Database

we use:

User Entity



UserResponse DTO



Client

Entity vs DTO

Entity	DTO
Represents database table	Represents API data
Used by JPA/Hibernate	Used for data transfer
Contains database-related information	Contains only required API information
Should not normally be exposed directly	Safe to expose required fields
Example: User	Example: UserResponse
Contains sensitive/internal fields if needed	Can hide sensitive fields

Example

Think about your house.

Entity = Everything inside your house

DTO = Things you choose to show to a visitor

You don't give a visitor access to everything in your house.

Similarly, you don't expose your complete database Entity to an API client.

Request DTO

A **Request DTO** contains the data that the client is allowed to send to your API.

Example:

```
public class CreateUserRequest {  
    private String name;
```

```
private String email;

private String password;

}
```

The client sends:

```
{

  "name": "Ravi",

  "email": "ravi@gmail.com",

  "password": "abc123"

}
```

The controller receives:

```
@PostMapping

public UserResponse createUser(

    @RequestBody CreateUserRequest request) {

    return userService.createUser(request);

}
```

Response DTO

A **Response DTO** contains the information that your API wants to send back to the client.

```
public class UserResponse {

    private Long id;

    private String name;

    private String email;

}
```

Notice:

User Entity

id

name

email

password

but:

UserResponse

id

name

email

The password is not exposed.

The API response becomes:

```
{  
  "id": 1,  
  "name": "Ravi",  
  "email": "ravi@gmail.com"  
}
```

This is one of the major reasons we use DTOs.

Step 2 — Create DTOs

Create CreateUserRequest DTO

Create:

dto/CreateUserRequest.java

This DTO is used when creating a new user.

```
package com.example.userservice.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

public class CreateUserRequest {

    @NotBlank(message = "Name is required")

    @Size(min = 2, max = 50,
        message = "Name must be between 2 and 50 characters")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Email must be valid")
    private String email;

    @NotBlank(message = "Phone is required")
    @Pattern(
        regexp = "^[6-9]\\d{9}$",
        message = "Phone must be a valid 10 digit Indian mobile number"
    )
    private String phone;

    @NotBlank(message = "Password is required")
    @Size(min = 8, max = 20,
        message = "Password must be between 8 and 20 characters")
    private String password;

    public String getName() {
        return name;
    }
}
```

```
public void setName(String name) {  
    this.name = name;  
}  
public String getEmail() {  
    return email;  
}  
public void setEmail(String email) {  
    this.email = email;  
}  
public String getPhone() {  
    return phone;  
}  
public void setPhone(String phone) {  
    this.phone = phone;  
}  
public String getPassword() {  
    return password;  
}  
public void setPassword(String password) {  
    this.password = password;  
}  
}
```

Create UpdateUserRequest

Create:

dto/UpdateUserRequest.java

```
package com.example.userservice.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

public class UpdateUserRequest {

    @NotBlank(message = "Name is required")

    @Size(min = 2, max = 50,
        message = "Name must be between 2 and 50 characters")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Email must be valid")
    private String email;

    @NotBlank(message = "Phone is required")
    @Pattern(
        regexp = "^[6-9]\\d{9}$",
        message = "Phone must be a valid 10 digit Indian mobile number"
    )
    private String phone;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
```

```
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPhone() {
        return phone;
    }

    public void setPhone(String phone) {
        this.phone = phone;
    }
}
```

Create UserResponse DTO

Create: dto/UserResponse.java

```
package com.example.userservice.dto;

public class UserResponse {
    private Long id;
    private String name;
    private String email;
    private String phone;
    public UserResponse() {
    }

    public UserResponse(Long id, String name,
        String email, String phone) {
        this.id = id;
```

```
        this.name = name;

        this.email = email;

        this.phone = phone;
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getPhone() {
        return phone;
    }
}
```

Step 3 — Remove direct Entity exposure

Use DTO

Create a DTO specifically for the API request.

```
public class CreateUserRequest {

    private String name;

    private String email;

    private String phone;

    private String password;
}
```

Then the controller receives the DTO:

```
@PostMapping("/users")  
  
public UserResponse createUser(  
    @RequestBody CreateUserRequest request) {  
    return userService.createUser(request);  
}
```

So the flow becomes:

Client



CreateUserRequest DTO



Controller



Service



User Entity



Database

And when returning data:

Database



User Entity



Service



UserResponse DTO



Client

Step 4 — Add Validation

Now we need to make sure the client sends **correct data**.

@NotBlank

@NotNull

@Email

@Size

@Pattern

Example:

```
public class CreateUserRequest {  
    @NotBlank(message = "Name is required")  
    private String name;  
    @NotBlank(message = "Email is required")  
    @Email(message = "Invalid email format")  
    private String email;  
    @NotBlank(message = "Phone is required")  
    @Pattern(regexp = "^[6-9][0-9]{9}$",message = "Invalid phone number")  
    private String phone;  
  
    @NotBlank(message = "Password is required")  
    @Size(  
        min = 8,  
        message = "Password must contain at least 8 characters"  
    )  
    private String password;  
}
```

Step 5 — Test invalid requests

Now we need to test whether our validation actually works.

Example, send this request from **Postman**:

```
{  
  "name": "",  
  "email": "abc",  
  "phone": "123",  
  "password": "123"  
}
```

This request contains multiple errors.

Validation results

name

""

↓

@NotBlank

↓

Name is required

email

"abc"

↓

@Email

↓

Invalid email format

phone

"123"

↓

@Pattern

↓

Invalid phone number

password

"123"

↓

@Size(min = 8)

↓

Password must contain at least 8 characters

The request should **not reach the service/database**.

simple example

DTO

```
package com.example.user.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

public class CreateUserRequest {

    @NotBlank(message = "Name is required")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;

    @NotBlank(message = "Phone is required")
    @Pattern(
        regexp = "^[6-9][0-9]{9}$",
        message = "Invalid phone number"
    )
    private String phone;

    @NotBlank(message = "Password is required")
```

```
@Size(min = 8,message = "Password must contain at least 8 characters")
private String password;
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public String getEmail() {
    return email;
}
public void setEmail(String email) {
    this.email = email;
}
public String getPhone() {
    return phone;
}
public void setPhone(String phone) {
    this.phone = phone;
}
public String getPassword() {
    return password;
}
public void setPassword(String password) {
    this.password = password;
}
}
```

Controller

```
@RestController
@RequestMapping("/users")
public class UserController {
    @PostMapping
    public UserResponse createUser(
        @Valid @RequestBody CreateUserRequest request) {
        return userService.createUser(request);
    }
}
```

Valid request

```
{
  "name": "Ravi",
  "email": "ravi@gmail.com",
  "phone": "9876543210",
  "password": "password123"
}
```

Validation passes → Controller → Service → Database.

Invalid request

```
{
  "name": "",
  "email": "ravi",
  "phone": "123",
  "password": "123"
}
```

Validation fails → Error response.

Step 6 — Create custom exceptions

UserNotFoundException

Create: exception/UserNotFoundException.java

```
package com.example.userservice.exception;

public class UserNotFoundException extends RuntimeException {

    public UserNotFoundException(String message) {
        super(message);
    }
}
```

DuplicateEmailException

```
package com.example.userservice.exception;

public class DuplicateEmailException extends RuntimeException {

    public DuplicateEmailException(String message) {
        super(message);
    }
}
```

InvalidUserException

```
package com.example.userservice.exception;

public class InvalidUserException extends RuntimeException {

    public InvalidUserException(String message) {
        super(message);
    }
}
```

Step 7 — Create Global Exception Handler

Handle all errors in one place instead of writing try-catch blocks inside every controller.

```
package com.example.userservice.exception;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.dao.DataIntegrityViolationException;
```

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import java.time.LocalDateTime;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GlobalExceptionHandler {

    // 1. Validation errors
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationException(
        MethodArgumentNotValidException ex,
        HttpServletRequest request) {
        String message = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .map(error ->
                error.getField() + ": " +
                error.getDefaultMessage())
            .collect(Collectors.joining(", "));

        ErrorResponse errorResponse = new ErrorResponse(
            LocalDateTime.now(),
            400,
            "VALIDATION_ERROR",
            message,
            request.getRequestURI()
        );
    }
}
```

```

return new ResponseEntity<>(errorResponse,HttpStatus.BAD_REQUEST);
}

// 2. Resource not found
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse> handleResourceNotFoundException(
    ResourceNotFoundException ex,
    HttpServletRequest request) {
    ErrorResponse errorResponse = new ErrorResponse(
        LocalDateTime.now(),
        404,
        "USER_NOT_FOUND",
        ex.getMessage(),
        request.getRequestURI()
    );
    return new ResponseEntity<>(
        errorResponse,
        HttpStatus.NOT_FOUND
    );
}

// 3. Duplicate data
@ExceptionHandler(DataIntegrityViolationException.class)
public ResponseEntity<ErrorResponse> handleDuplicateData(
    DataIntegrityViolationException ex,
    HttpServletRequest request) {
    ErrorResponse errorResponse = new ErrorResponse(
        LocalDateTime.now(),
        409,
        "DUPLICATE_DATA", "Data already exists",request.getRequestURI() );
}

```

```

        return new ResponseEntity<>(errorResponse,HttpStatus.CONFLICT);
    }
// 4. Generic exception
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGenericException(Exception ex,
        HttpServletRequest request) {
        ErrorResponse errorResponse = new ErrorResponse(
            LocalDateTime.now(),
            500,
            "INTERNAL_SERVER_ERROR",
            "Something went wrong",
            request.getRequestURI()
        );
        return new ResponseEntity<>(
            errorResponse,
            HttpStatus.INTERNAL_SERVER_ERROR
        );
    }
}

```

Step 8 — Create Standard Error Response

```

{
    "timestamp": "2026-09-01T10:30:00",
    "status": 404,
    "error": "USER_NOT_FOUND",
    "message": "User not found",
    "path": "/api/v1/users/10"
}

```

We should create a Java class for this format.

ErrorResponse.java

```
package com.example.userms.response;

import java.time.LocalDateTime;

public class ErrorResponse {

    private LocalDateTime timestamp;

    private int status;

    private String error;

    private String message;

    private String path;

    public ErrorResponse(

        LocalDateTime timestamp,

        int status,

        String error,

        String message,

        String path) {

        this.timestamp = timestamp;

        this.status = status;

        this.error = error;

        this.message = message;

        this.path = path;

    }

    public LocalDateTime getTimestamp() {

        return timestamp;

    }

    public int getStatus() {

        return status;

    }

}
```

```
public String getError() {  
    return error;  
}  
public String getMessage() {  
    return message;  
}  
public String getPath() {  
    return path;  
}  
}
```

It defines the **common format** for every error.

timestamp → When error occurred

status → HTTP status code

error → Error type

message → Human-readable message

path → API URL where error happened

Step 9 — Test Every Error

Prove that your Global Exception Handler is working correctly.

Test 1 — 400 Bad Request

400 Bad Request means:

The client sent invalid data.

Example, your DTO might have:

@NotBlank(message = "Name is required")

private String name;

@Email(message = "Invalid email format")

private String email;

@Size(min = 8, message = "Password must be at least 8 characters")

private String password;

Send an invalid request in Postman:

POST http://localhost:8080/api/v1/users

Body → raw → JSON:

```
{
  "name": "",
  "email": "abc",
  "phone": "123",
  "password": "123"
}
```

Expected response

```
{
  "timestamp": "2026-09-03T14:30:00",
  "status": 400,
  "error": "VALIDATION_ERROR",
  "message": "name: Name is required, email: Invalid email format, phone: Invalid phone number, password: Password must be at least 8 characters",
  "path": "/api/v1/users"
}
```

We capture

POST request

+

Invalid JSON

+

400 response

This becomes your **400 test evidence**.

Test 2 — 404 Not Found

Now test the custom `ResourceNotFoundException`.

Suppose your database has users:

ID

1

2

3

Try requesting:

```
GET http://localhost:8080/api/v1/users/10
```

If user 10 doesn't exist, your service should throw:

```
throw new ResourceNotFoundException("User not found");
```

The global handler catches it.

Expected response

```
{  
  "timestamp": "2026-09-03T14:30:00",  
  "status": 404,  
  "error": "USER_NOT_FOUND",  
  "message": "User not found",  
  "path": "/api/v1/users/10"  
}
```

We capture

```
GET /api/v1/users/10
```

+

404 response

Test 3 — 409 Duplicate Data

This tests duplicate information.

For example, suppose your email is unique:

```
@Column(unique = true)
```

```
private String email;
```

Database already contains:

```
ravi@gmail.com
```

Now send: POST http://localhost:8080/api/v1/users

```
{  
  "name": "Ravi",  
  "email": "ravi@gmail.com",  
  "phone": "9876543210",  
  "password": "password123"  
}
```

Because the email already exists, the database can generate:

DataIntegrityViolationException

Your global handler catches it.

Expected response

```
{  
  "timestamp": "2026-09-03T14:30:00",  
  "status": 409,  
  "error": "DUPLICATE_DATA",  
  "message": "Data already exists",  
  "path": "/api/v1/users"  
}
```

Test 4 — 500 Internal Server Error

This tests your generic exception handler.

Your handler contains:

```
@ExceptionHandler(Exception.class)  
public ResponseEntity<ErrorResponse> handleGenericException(  
    Exception ex,  
    HttpServletRequest request) {  
    ErrorResponse errorResponse = new ErrorResponse(  
        LocalDateTime.now(),  
        500,
```

```

        "INTERNAL_SERVER_ERROR",
        "Something went wrong",
        request.getRequestURI()
    );
    return new ResponseEntity<>(
        errorResponse,
        HttpStatus.INTERNAL_SERVER_ERROR
    );
}

```

If an unexpected exception occurs, it will return:

```

{
    "timestamp": "2026-09-03T14:30:00",
    "status": 500,
    "error": "INTERNAL_SERVER_ERROR",
    "message": "Something went wrong",
    "path": "/api/v1/users"
}

```

For testing, you can temporarily create an endpoint such as:

```

@GetMapping("/test-error")
public String testError() {

    throw new RuntimeException("Testing generic exception");
}

```

Then call: GET <http://localhost:8080/api/v1/users/test-error>

Expected:

500 Internal Server Error

After testing, you can remove the temporary endpoint.

You should be able to show these four tests:

Test	API scenario	Expected
1	Invalid DTO data	400
2	User doesn't exist	404
3	Duplicate email/data	409
4	Unexpected exception	500

Step 10 — Document

Add error-response examples to README

User Service

Error Handling

The application uses a global exception handler using `@RestControllerAdvice`.

400 - Validation Error

```
{
  "timestamp": "2026-09-03T14:30:00",
  "status": 400,
  "error": "VALIDATION_ERROR",
  "message": "Email: Invalid email format",
  "path": "/api/v1/users"
}
```

404 - User Not Found

```
{
  "timestamp": "2026-09-03T14:30:00",
  "status": 404,
  "error": "USER_NOT_FOUND",
  "message": "User not found",
  "path": "/api/v1/users/10"
}
```

409 - Duplicate Data

```
{  
  "timestamp": "2026-09-03T14:30:00",  
  "status": 409,  
  "error": "DUPLICATE_DATA",  
  "message": "Data already exists",  
  "path": "/api/v1/users"  
}
```

500 - Internal Server Error

```
{  
  "timestamp": "2026-09-03T14:30:00",  
  "status": 500,  
  "error": "INTERNAL_SERVER_ERROR",  
  "message": "Something went wrong",  
  "path": "/api/v1/users"  
}
```